

The myth of accelerating business processes through parallel job designs

Michael Zapf · Ute Lindheimer · Armin Heinzl

Published online: 22 September 2006
© Springer-Verlag 2006

Abstract This paper discusses postulated advantages of parallel job designs compared with sequential designs in business processes. Until now, the literature suggests an overall dominance of parallel designs. An analysis of relevant application domains, in which parallel tasks are prevalent in order to accelerate processes, offers two remarkable insights: (1) parallel tasks can but do not have to speed-up business processes; (2) coordination efforts may reduce or even invert potential performance gains. Thus, this paper thoroughly evaluates interdependencies between parallel designs and coordination efforts through simulation experiments. Several process patterns for order processing in a real-world example will be examined. The results offer noteworthy characteristics of parallel designs which extend the existing knowledge: (a) gains from parallelization are reduced with increasing process variability, (b) small increments in coordination efforts neutralize gains from parallelization, (c) in high work load situations, the resource capacity for the coordination task is a severe bottleneck, and (d) if the findings (a)–(c) are carefully considered, only the parallelization of multiple tasks leads to significant performance gains.

Keywords Business process design · Coordination theory · Parallel designs · Job parallelization · Order processing · Process simulation

M. Zapf
REHAU AG, Rehau, Germany
e-mail: michael.zapf@rehau.com

U. Lindheimer
Accenture Germany, Munich, Germany
e-mail: ute.lindheimer@accenture.com

A. Heinzl (✉)
Information Systems I, University of Mannheim, Mannheim 68131, Germany
e-mail: aheinzl@uni-mannheim.de

1 Introduction

Business process management literature postulates parallel job processing as an important approach to speed up business processes (e.g. Cuiper et al. 1996; Davenport 1993; Edosomwan 1996; Eversheim 1995; Goebel 1996; Hammer and Champy 1993; Nissen 1998; Ould 1995; Schmelzer 1990; Zangl 1985). General guidelines combined with success stories are used to deliver the wide-spread message: a parallelization of tasks reduces the overall throughput time! This notion seems to be compelling at a first glance but cannot be confirmed without limitations on a closer view. Some authors state that parallel process designs increase processing speed but may result in a higher coordination effort which accordingly reduces expected efficiency gains (e.g. Achermann 1998; Adler et al. 1995; Davenport 1993; Hammer 1990; Zangl 1985). Unfortunately, no empirical evidence is offered for the predominance of parallel designs. Furthermore, no insight is offered under which circumstances the gains from parallel job processing may be offset by induced additional coordination efforts.

In this paper, we will provide a profound analysis of the interdependence between performance gains through parallel designs and the coordination effort. The impact of processing time variability and work load intensity will be evaluated in detail.

The following study focuses on the deployment of multiple employees who parallel tasks for the same job. In this context, additional coordination activities are necessary for reconciling the partial results of parallel tasks (see Adler et al. 1995; AitSahlia et al. 1995; Cuiper et al. 1996; Walter 1998). Another way of parallel job processing—which is not within the scope of this paper—would be the assignment of similar jobs to different resources who work in parallel on their tasks (see Blocher 1996; Bukchin and Rubinovitz 2003; Buzacott 1990, 1996; Johnson 1983; Piersma and Romeijn 1996; Pinto et al. 1975; Sheu and Babbar 1996; Seidmann and Sundararajan 1997). In this classical problem of queuing theory, resources perform all tasks for one job and no intra-job coordination is necessary. The resource capacity is enhanced for a business process but not for a single job.

The paper is structured as following: first, the existing literature base is reviewed with respect to relevant application domains where parallel process designs prevail. Empirical studies and analytical models are of special interest because they offer a clearer understanding of gains and limitations through parallelization. In addition, relevant literature from coordination theory will be discussed in order to demonstrate the causes and effects of coordination activities within organizations.

Subsequently, generic parallel design scenarios will be developed for order processing based on a real-world example. These scenarios are modeled as stochastic processing networks and appear to be suitable for an exemplification of the effects of parallel designs and associated coordination activities. Then, model parameters, performance measures, and an evaluation scheme will be introduced in order to derive and discuss the results of our simulation

experiments. At the end of this paper, valuable managerial conclusions will be outlined and future research directions are suggested.

2 Related contributions

2.1 Business process management

As outlined above, the existing literature on business process management discusses parallel job processing cursory. General guidelines are given for designing business processes and the parallelization of tasks is regarded as one way to improve the efficiency of the overall throughput time of business processes. Some authors like Ould (1995, p. 157), for instance, offer simple reasoning for demonstrating parallelization gains in a static environment: The throughput time ($t_A + t_B$) of two task A and B in the sequential design may be reduced to a throughput time of $\max(t_A, t_B)$ in a parallel design, where t_i denotes the processing time of task i . Obviously, numerous other authors seem to have this reasoning in mind when they postulate parallel job processing as desirable whilst addressing the intuitive understanding of their readers (e.g. Cuiper et al. 1996; Edosomwan 1996; Eversheim 1995; Goebel 1996; Nissen 1998; Schmelzer 1990; Zangl 1985). In a few instances, business cases are provided in order to further provide (face) validity of the suitability of parallel process designs. Unfortunately, these examples do not disclose the basis for this reasoning as well as the empirical data for the argumentation (e.g. Davenport 1993; Hammer and Champy 1993). Thus, the myth of parallel designs has been developed on loose ground.

Nevertheless, some contributions confess that parallel designs cause a more complex process structure which leads to increasing coordination efforts (e.g. Achermann 1998; Adler et al. 1995; Davenport 1993; Hammer 1990; Zangl 1985). This coordination effort may reduce the process efficiency and requires additional resources. One important factor which seems to level this compensational effect is the utilization of information technology for the automation of coordination tasks (e.g. Davenport 1993; Hammer 1990; Walter 1998).

2.2 Product development and simultaneous engineering

An in-depth discussion of parallel job processing can be found in the product development domain; especially the discussions about simultaneous engineering. Within this approach, the product development time should be reduced through overlapping and parallel development tasks (see Gerpott and Winzer 1996). Table 1 lists selected studies which examine the success of simultaneous engineering. The references are categorized concerning the applied method, scope and stated paralleling gains.

The majority of the listed publications state the effect of parallel tasks as positive regarding the reduction of development time. But the achievable

Table 1 Studies regarding parallel job processing in the product development literature

References	Method	Scope	Parallellization gains
AitSahlia et al. (1995) Clark and Fujimoto (1991)	Analytical model Empirical study (interviews)	20 companies from US, Western Europe and Japan, 29 projects (automobile)	Positive, not quantified Positive, not quantified
Griffin (1993) Handfield (1994)	Empirical study (secondary) Empirical study (interviews)	21 companies mainly from US 31 companies from US and Canada (job production)	-53 % of development time -41 % of development time, -49 % of time to delivery for new products, +76 % of time to delivery for developed products Positive, not quantified
Kessler and Chakrabarti (1999) Murmman (1994)	Empirical study (interviews) Empirical study (interviews)	10 US companies, 75 projects 8 German companies, 14 projects (mechanical engineering)	-25 % of development time Positive, not quantified -2 % of development time
Schröder (1994) Trygg (1993)	Analytical model Empirical study (interviews)	109 Swedish companies (mechanical engineering and metal-processing) 12 German companies	From -30 to -50 % of development time

gains are valued extremely different: Starting from 2% (Trygg 1993) gains go up to 53% (Griffin 1993). Some authors do not value parallelization gains and Handfield (1994) mentions even a performance loss of 76% regarding the time to delivery for an enhancement of already developed products.

These results provide further evidence that parallel job designs in simultaneous engineering may lead to performance gains but do not guarantee them. In this context, the interpretation of the numerical results has to be conducted very carefully since (a) the studies examine the effects of simultaneous engineering from a holistic perspective and not from a process perspective and (b) the method of data collection is largely based on interviews which may cause uncontrolled settings in which parallelization gains cannot be analyzed in detail.

2.3 Parallel computing

Parallel job processing is an essential part of parallel computing. The execution of computer programs is accelerated through different types of parallelism in this domain which can be classified as follows (see Heiss 1994):

- (a) Internal parallelism within one program (intra-program parallelism) and
- (b) External parallelism between multiple programs (inter-program parallelism).

The intra-program parallelism takes advantage of the fact that computer programs contain not only sequential execution and control structures but also parallel control structures. The achievable performance gains depend on the number of parallel processors which are utilized for parallel tasks and the communication time. Figure 1 shows the relation between the number of processors n and overall processing time $T(n)$. $T_x(n)$ denotes the

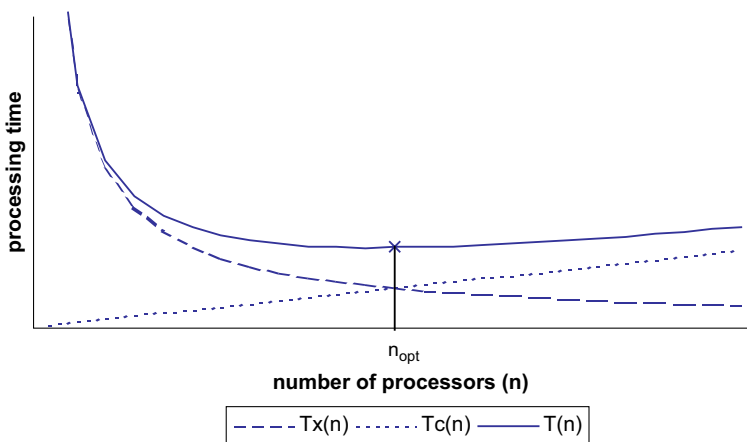


Fig. 1 Relation between number of processors and processing time for intra-program parallelism (see Heiss 1994, p. 51)

execution time which is needed for executing the program instructions dependent on the number of processors n . It results from Amdahls law as a monotonous decreasing curve since the achievable speed-up through paralleling is limited through sequential data dependencies which cannot be parallelized (see Heiss 1994; Amdahl 1967). As parts of a parallel program require interaction with others, some coordination has to take place between the single parts. The accompanying coordination time $T_c(n)$ can be modeled as a strict monotonous increasing curve with the number of processors. The overall processing time $T(n)$ results as the sum of execution and coordination time:

$$T(n) = T_x(n) + T_C(n).$$

The utilization of too many processors leads to an increase of processing time which is presented in Fig. 1 for $n > n_{\text{opt}}$. This effect is called processor thrashing and expresses the fact that an increasing coordination effort occupies too many processors with unproductive work (see Heiss 1994). The trade-off between gains from parallelization and communication effort results in an optimum number of processors n_{opt} with minimum processing time. In extreme cases, the communication effort prohibits parallel processing which has been shown in model calculations by Stone (1987).

The inter-program parallelism deals with multiple programs which are executed in parallel on a shared amount of processors. The program instructions have to be distributed among the processors. According to the distribution strategy, the gains from parallelization of intra-program parallelism may be reduced in order to achieve a better utilization of the processor resources (see Heiss 1994).

In summary, the parallel computing literature states that parallel program execution leads to significant performance gains if (a) the assignment of different tasks on parallel processors is done efficiently and (b) no significant coordination effort has to be considered (see Heiss 1994; IEEE 1995; Siemers 1998). If parallel computing needs a costly intra-program or inter-program coordination, the overall system has either to be designed very carefully (see Glücher 1998; Jaenicke 1998) or the performance gains may be limited or even reversed (see Hächler 1998; Smith 1999).

2.4 Coordination within organizations

The previous discussion of various application domains has shown that significant performance gains can be achieved through parallel job processing. But it has also become apparent that the performance of parallel designs depends strongly on the accompanying coordination effort. As the effect of coordination within organizations has been widely discussed in organizational literature, we will provide a short overview of selected work from this field and transfer the statements to the application of parallel job processing.

Malone and Crowston (1994, p. 90) defines coordination as “managing dependencies between activities”. In the case of parallel business processes, dependencies may exist between paralleled tasks respectively their outcomes. Special activities are necessary in order to manage coordination which Crowston (1997, p. 169) calls coordination mechanisms. These additional activities point to additional effort within a process and are often called synchronization activities in parallel process designs. Malone and Smith (1988, p. 10) names the accompanying effort as coordination costs and uses the time as primary measure. He also assumes that coordination costs increase with the number of communication links. As parallel designs have more links than sequential designs, this would lead to the hypothesis that the coordination costs in parallel designs tend to be higher than the costs in a corresponding sequential design.

The connection between communication links (dependencies) and coordination costs refers to the work of Thompson (1967). He differentiates between three types of dependencies within an organization: (a) pooled, (b) sequential and (c) reciprocal dependencies. The communication links and accompanying coordination effort increases from (a) to (c). With regard to the performance of parallel designs, this means that it is better if the dependencies within the process are pooled or sequential rather than being reciprocal. If the dependencies are reciprocal, they should be performed with one organizational unit in order to avoid additional coordination costs across organizational boundaries. Kilman (1983) presents an approach for re-engineering the structure of an organization according to the task dependencies. The high coordination effort between different organizational units has also been stated by Barua and Ravindran (1996) who developed an analytical model for the information sharing between organizational units.

Housel and Kanevsky (1995) discusses the relation between process costs and process outcome. He suggests that a process should only consist of tasks which contribute to a value-add. As a consequence, the question arises whether coordination activities contribute a value-added and whether a parallel task design makes sense if additional coordination effort arises.

2.5 Pre-conclusion

The utilization of parallel job processing in different domains shows (a) that parallel designs can be more efficient than sequential designs and (b) the

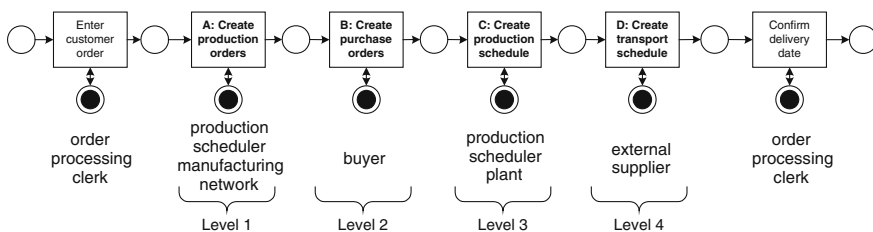


Fig. 2 Scenario I: sequential order processing with four levels

induced coordination efforts play an important role for the performance and predominance of parallel designs. This interdependency will be analyzed in the following by developing various scenarios for order processing within an industrial enterprise.

3 Parallel design scenarios for order processing within an industrial enterprise

In our simulation study, we refer to a real-world example of an order processing workflow in the construction supply industry. The enterprise offers complex building systems to commercial customers, produces some of the components itself and purchases other components from an external supplier. We will develop and utilize three different variations (scenarios) for the order processing workflow as the starting point of our analysis.

The first scenario describes the sequential processing of a customer order. It is used as reference base for the analysis of the following parallel processing models. The corresponding business process is presented as a Petri Net in Fig. 2.

First, the incoming order is accepted by an order processing clerk. He enters the order, performs a credit assessment and forwards the order to the production scheduler for the manufacturing network. The scheduler selects the production plants, determines the delivery dates for in-house production and creates the production orders. Now, the order is passed on to the buyer who selects the external suppliers, determines the delivery dates for the purchased parts and creates the purchase orders. Based on the specified delivery dates, the production scheduler of the selected plant creates the production schedule. Afterwards, the external supplier creates the transport schedule for the purchased parts and the order processing clerk confirms the delivery date to the customer.

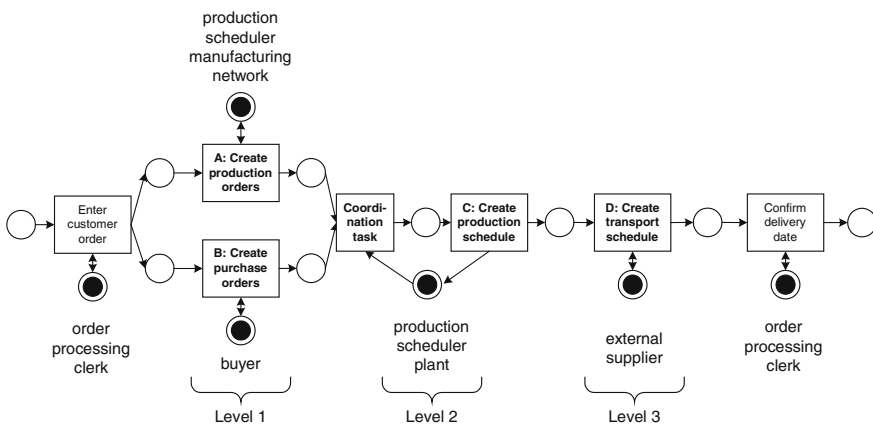


Fig. 3 Scenario II: parallel order processing with three levels

Two alternative design patterns will be derived from this base scenario with different degrees of parallelism. The following premises have to be kept in mind for the resulting process models:

- Processing order: we assume that production orders have to be created (task A) before creating the production schedule (task C). Moreover, the transport schedule (task D) is dependent on the creation of purchase orders (task B). Therefore, A has to be performed before C and in addition B has to be performed before D.
- One activity is performed by one person. No additional coordination takes place between the departments.
- One person performs one activity at a certain point in time and does not evaluate multiple requests simultaneously.
- Requests are handled according to the First In-First Out (FIFO) strategy. Activities which have started once will not be interrupted by other requests.

In the first step, we introduce Scenario II that emerges from scenario I through parallelizing the tasks A and B (see Fig. 3). The order processing clerk forwards the customer order in parallel to the production scheduler (task A) and buyer (task B). After creating the production and purchase orders, the production scheduler in the plant has to synchronize the delivery dates (additional coordination task) and create the production schedule (task C). The subsequent process structure remains the same as in scenario I. With this scenario, a reduction of the processing time is expected.

The third scenario III implies a further parallelization of activities: The creation of production orders (task A) and the creation of the production

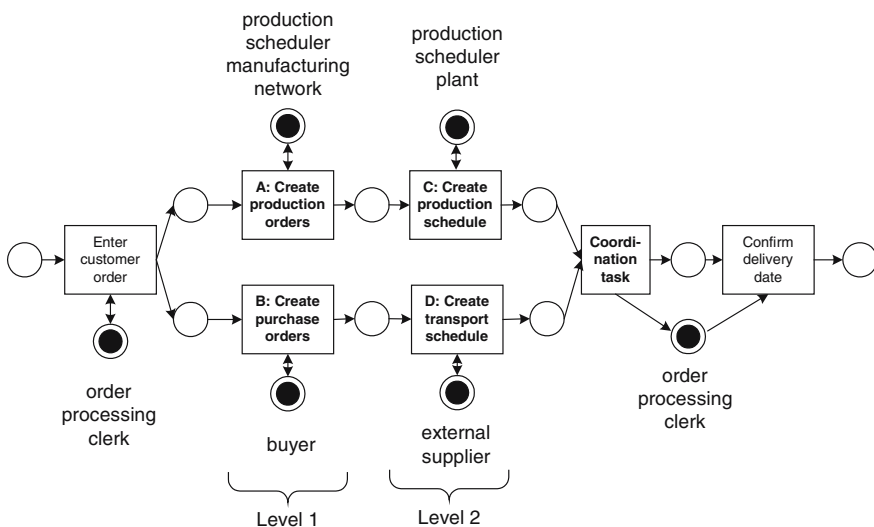


Fig. 4 Scenario III: parallel order processing with two levels

schedule (task C) are performed in parallel to the creation of purchase orders (task B) and the creation of the transport schedule (task D). This leads to a process with two levels as presented in Fig. 4. An additional coordination task is necessary for synchronizing the production schedule and the transport schedule.

4 Methodology

We apply an experimental approach based on computer simulation for evaluating the paralleling scenarios described above. This kind of research design has been widely and successfully used for evaluating business process designs (e.g. Adler et al. 1995; Choi and Kin 1998; De Vreede et al. 1996; Farrington and Nazemetz 1998; Giannini et al. 1997; Prietula and Carley 1994; Shanker and Tzen 1985; Zapf and Heinzl 2000; Zapf and Storch 2001; Zapf 2001).

As parallel process designs are non-static discrete systems, they will be represented as discrete event computer models and analyzed with the help of stochastic simulation (see Law et al. 1991). For preparing the simulation model and performing the experiments, the simulation tool ARENA is used (Kelton et al. 1998). The experiments are performed in the form of steady-state simulations in order to analyze the long-term effect of parallel designs. Every simulation experiment consists of 1,000 independent runs. This number is much higher than general rules from literature and ensures the validity of the obtained results (Bulgren 1982 suggests for example 30 independent runs). Every run represents 2,000 min of work which corresponds to 4 days with 8 (1/3) working hours per day). The warm-up period of 3,000 min has not been included in the results.

5 Model parameters and performance measures

An overview of the model parameters is presented in Table 2. The inter-arrival time represents the time between the arrivals of two orders. As the inter-arrival time varies significantly in our real-world process, we model it as stochastic parameter and use the exponential distribution which has been frequently proposed for incoming customer orders in the literature (see

Table 2 Model parameters

Parameter	Description	Characteristic
Inter-arrival time	Time between the arrival of two different requests	Deterministic, stochastic
Handling time	Time for executing a task for one request	Deterministic, stochastic
Coordination factor	Models the additional effort for the synchronization of parallel tasks, the factor is multiplied with the processing time of the synchronization activity	Deterministic

Kelton et al. 1998; Liebl 1995; Sheu and Babbar 1996; Buzacott 1996; Seidmann and Sundararajan 1997). However, in the first experimental series, we assume the inter-arrival time as deterministic parameter in order to obtain a suitable reference basis for our experimental study.

The handling (processing) time is required for executing one single task. It is dependent on the task type (production order, purchase order, production schedule, transport schedule) and the complexity of the current order. In order to evaluate the impact of the handling time on the parallelization gains, we model the handling time alternatively as (a) deterministic and (b) stochastic parameter with a triangular distribution. We use the triangular distribution since the exact form of the distribution is not known but we can guess the minimum, maximum and most likely value (see Kelton et al. 1998). Two different experimental series are separated in the stochastic case: (b1) balanced system and (b2) imbalanced system. In a balanced system, the handling times of all tasks in the process have the same average values. Within an imbalanced system, the average handling times vary for all tasks.

The coordination factor is the central parameter for our analysis. It is introduced in order to reflect the coordination costs which are caused by an additional effort for coordinating parallel tasks. The explicit reflection of coordination costs within our analysis refers to the work of Thompson (1967), Kilman (1983) and Malone and Smith (1988) who explain the effects of coordination efforts between organizational units and show ways to reduce it (see paragraph 2).

The coordination factor is modeled as percentage of the handling time of the succeeding task. A high coordination effort (e.g. 180%) leads to a higher average handling time (e.g. 180% of the original handling time).

This study evaluates the influence of parallel designs on the processing speed and therefore uses the throughput time as the main performance measure. The throughput time begins with the arrival of a new order and ends after completing the final activity for it in the process. It consists of the

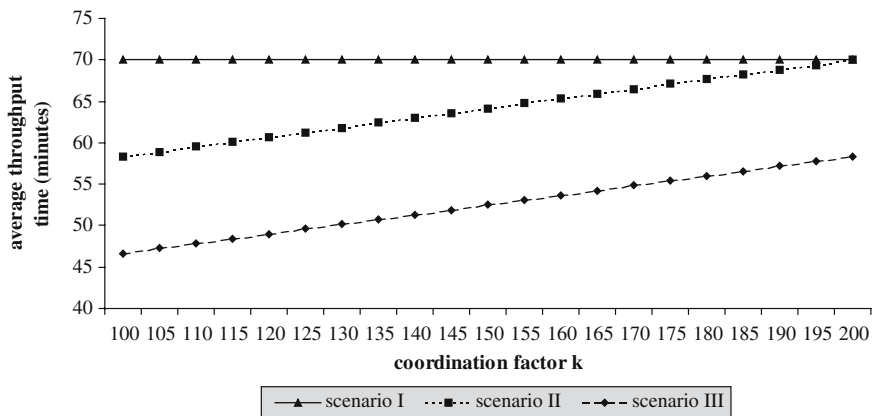


Fig. 5 Average throughput time for scenarios I–III in a deterministic environment with deterministic work load and processing time

Table 3 Experimental design and parameter values in minutes

Experimental series	Inter-arrival time	Processing time		
		Deterministic	Stochastic balanced	Stochastic imbalanced
Deterministic environment	>11:40	11:40	–	–
Processing variability, low work load	expo(45)	11:40	tria(3,8,24)	Task A: tria(1,4,12) Task B: tria(5,12,36) Others: tria(3,8,24)
Processing variability, medium work load	expo(25)	11:40	tria(3,8,24)	Task A: tria(1,4,12) Task B: tria(5,12,36) Others: tria(3,8,24)
Parallelism scope, medium work load	expo(25)	–	tria(3,8,24)	–
Parallelism scope, high work load	expo(14)	–	tria(3,8,24)	–

expo exponential distribution, *tria* triangular distribution

handling time and waiting time. Within the experiments, we measure the average processing time over all requests.

The availability of resources is crucial for the process performance. The throughput time, for example, increases exponentially dependent on the utilization of employees (see Kleinrock 1975). In order to evaluate parallelization gains in relation to the work load of employees, the average utilization of employees is measured in addition.

6 Numerical analysis

6.1 The deterministic starting point

The numerical analysis commences with a simple reasoning in order to take up the general statements of many authors who propose parallel job processing for accelerating business processes without any restrictions (see Sect. 1). Therefore, a static environment is assumed with a constant processing time of 11:40 min per task, a constant arrival rate and no turnaround time. This setting offers the maximum gain which can be achieved through parallel job processing.

Without any additional coordination effort ($k = 100$) the difference between scenario I and scenario II is 11:40 min which is exactly the processing time of one task (see Fig. 5). The break-even point for scenarios I and II is $k = 200$. In other words, the parallel job processing is more suitable than the sequential design as long as the additional coordination effort does not exceed the processing time of one complete task. The additional parallelization in scenario III further improves the process performance and yields an efficiency gain of 23:20 min in the best case ($k = 100$). The break-even point for scenario

I and III is $k = 300$ which is out of the scope of Fig. 5. These simple calculations seem to prove that parallel job processing may lead to a maximum speed-up of 17% for scenario II ($=1/6$ of the overall throughput time) and 33% for scenario III ($=2/6$ of the overall throughput time).

6.2 Experimental design

The assumption of a constant arrival rate and processing rate does not hold true in most practical environments (see Kelton et al. 1998). Therefore, we perform multiple experiments with different levels of variability regarding the inter-arrival time and the processing time. The analyzed variability levels and parameter constellations are listed in Table 3.

The inter-arrival time is modeled with the exponential distribution. The utilized distributions for three different workload situations (low, medium and high) are shown in Table 3 with the corresponding mean values. Expo(45) is used as an abbreviation for an exponential distributed inter-arrival time with a mean value of 45 min. In the deterministic environment, the inter-arrival time is not exactly specified, because it has no influence on calculating the throughput time as long as it is greater than the processing time of one task (11:40 min).

The triangular distribution is utilized within the stochastic environments for the processing time (see Kelton et al. 1998). For every triangular distribution, the minimum, mode and maximum values are presented in Table 3 and abbreviated as tria(minimum, mode, maximum). At this point, we emphasize that the mean value of the tria(3, 8, 24)-distribution calculates as $(3 + 8 + 24)/3 = 11:40$ min and thus corresponds to the deterministic environment. Within the stochastic imbalanced environment, task A has a shorter processing time and task B has a longer processing time (see Table 3). The corresponding

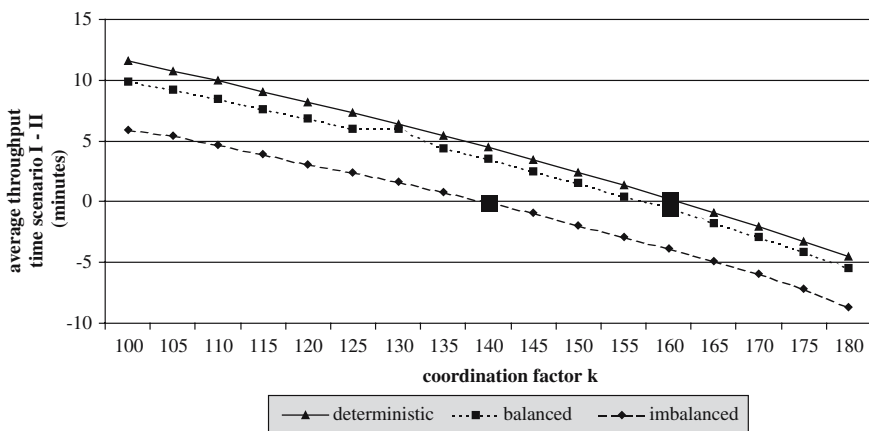


Fig. 6 Differences between the average throughput time of scenario I and II for different variability levels in a low work load environment

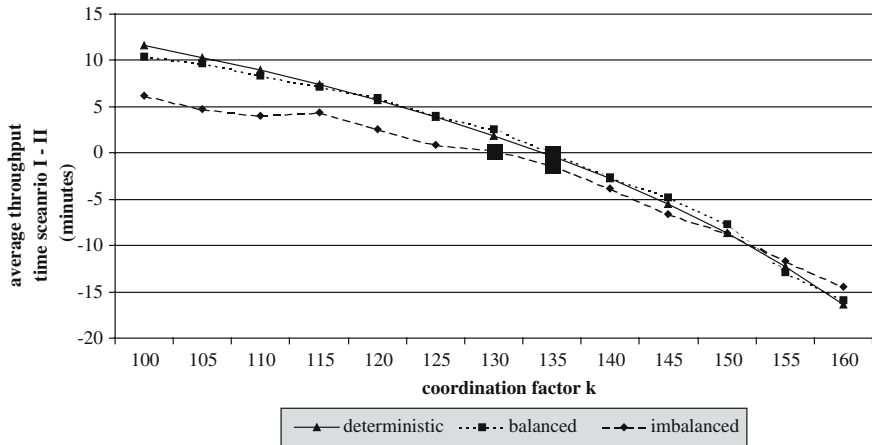


Fig. 7 Differences between the average throughput time of scenario I and II for different variability levels in a medium work load environment

mean values 5:40 min and 17:40 min sum up to 23:20 min which is twice as much as their deterministic processing time. Therefore, the sum of the processing time mean values remains the same over all environments.

6.3 The influence of processing time variability

In this section, we include the influence of processing time variability in our break-even analysis. The sequential reference scenario I is compared to scenario II which realizes the parallel design of two tasks and comprises three levels.

The requests arrive with an average inter-arrival time of 45 min in the “low work load” environment which leads to a low amount of work for the employees. The maximum average utilization of employees ranges between 39.04 and 41.32%. Figure 6 presents the differences between scenario I and scenario II with regard to the average throughput time for different coordination factors. A positive value indicates that parallel job processing is superior to the sequential design.

On the basis of a deterministic processing time and a stochastic arrival rate, the break-even point is $k = 160$ (deterministic). This demonstrates a substantial difference to the starting point with deterministic arrival rates where the break-even point was $k = 200$ (see Fig. 5) and indicates that the advantages of parallel designs decrease with the variability of the inter-arrival time.

The further increase of variability into the process causes decreasing efficiencies in scenario II. In the balanced environment with equal stochastic processing time per task, the performance of scenario II decreases slightly and the break-even point remains $k = 160$. Imbalanced tasks make the situation even worse. In this case, parallel job processing is inferior for coordination factors of $k > 140$.

The performance losses of scenario II with increasing variability can also be observed in the “medium work load” environment (see Fig. 7). This environment is characterized by an average inter-arrival time of 25 min which results in a maximum average utilization over all involved persons between 62.87 and 70.43%. In the deterministic and stochastic balanced environments, the parallel design scenario II is superior for $k < 135$. In the case of stochastic imbalanced processing time, the break-even point decreases to $k = 130$.

The experiments of this section show that a parallel process design performs better than a sequential one. But the advantages decrease due to an increasing level of processing time variability and due to increasing work load. These findings can be explained by the existence of an additional turnaround time in scenario II which arises if task A takes longer than task B or vice versa. The workflow does not continue unless both tasks have been finished and, therefore, always the longest processing time of two parallel tasks determines the overall throughput time.

If an additional coordination effort is necessary to synchronize the parallel tasks, the parallel processing may even be inferior to the sequential design. Depending on the processing time variability and the work load, the break-even coordination factor k lies between 130 and 160. Thus, in some cases, an additional coordination effort of 30% makes the sequential job design recommendable.

6.4 Parallelization scope and achievable efficiency gains

In the previous section, we examined the performance gains through the parallelization of two tasks with different levels of processing time variability. In this section, different scopes of parallel designs are compared with each

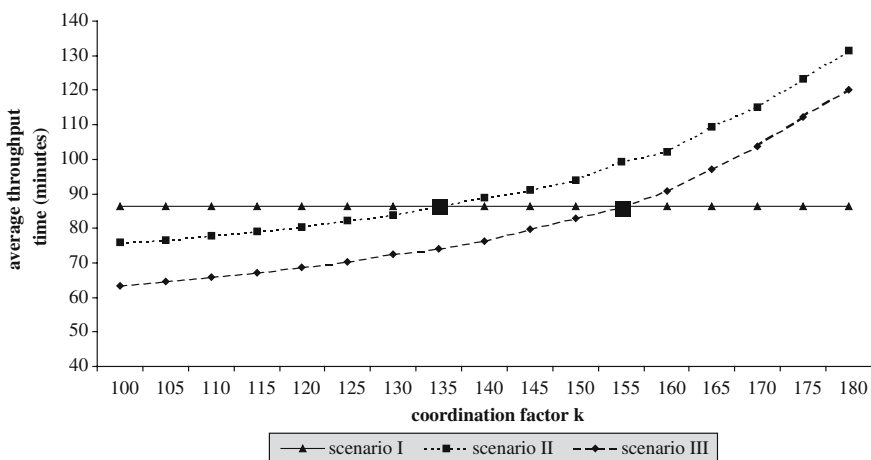


Fig. 8 Average throughput time for scenarios I–III with stochastic balanced processing time in a medium work load environment

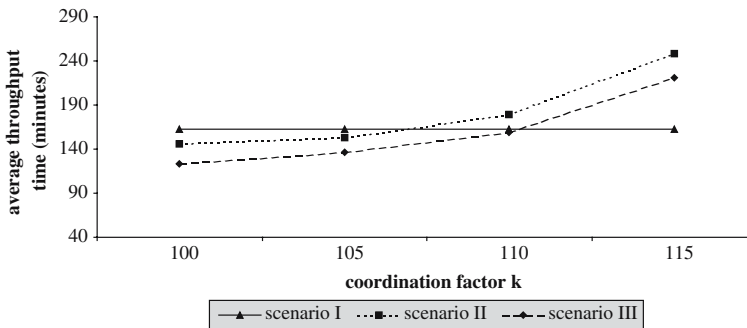


Fig. 9 Average throughput time for scenarios I–III with stochastic balanced processing time in a high work load environment

other in the stochastic balanced environment. First, the comparison is based on a medium work load with an average inter-arrival time of 25 min and a resulting maximum average utilization between 62.56 and 72.14%.

Figure 8 shows that the more extensive parallelism in scenario III reduces the average throughput time and moves the break-even point from $k = 135$ (scenario II) to $k = 155$ (scenario III). This indicates that the processing time variability can better be compensated in scenario III than in scenario II.

The second experimental series is performed under a high work load environment with an average inter-arrival time of 14 min. The resulting maximum average utilization of employees lies between 87.21 and 91.63% which reflects a typical overload situation.

Figure 9 shows that an additional coordination effort has strong influence on the favorability of parallel designs. A coordination effort of 10% or more makes the parallel job processing inferior to the sequential design. This can be explained with the extreme utilization of employees. Little increases of the processing time lead to disproportionate increases in the throughput time.

7 Discussion

Through the experimental analysis of idealized parallel designs, we were able to reveal new insights on parallel tasks as an option for (re-)designing business processes. The following conclusions should be taken into account when considering parallel job designs for the acceleration of business processes.

7.1 Variability reduces parallelization gains

In dynamic environments, the variability of the inter-arrival and processing time may cause an additional turnaround time. If job B arrives shortly after job A, B has to wait in the queue until the previous job A has been finished. If the processing time of job A takes longer than expected, the subsequent job B,

which has already arrived, has to wait in the queue. This offset turnaround time occurs in both sequential and parallel designs and may lead to an increasing average throughput time.

An additional turnaround time may be the case in parallel designs for two parallel tasks A and B. Even if both tasks have the same *average* processing time, this does not mean that they have exactly the same processing time for one specific job. If task A is completed but task B is not, the job cannot be processed until task B has also been finished. This holds true whether the succeeding employee is free or busy. Thus, for jobs with this characteristic, an additional turnaround time and, for employees, an additional idle time will be introduced through a parallel design.

These effects lead to the hypothesis that the performance gains through parallel designs are lower in dynamic environments than in static environments. Thus, it is important for the process designer to analyze the variability of the process environment in detail. Important impact factors are the inter-arrival time, the processing time and the balance of tasks.

7.2 Additional coordination effort slows down job processing more than expected

The performance gains of parallel designs are strongly dependent on the coordination effort which is necessary for synchronizing the results of parallel tasks. Within a break-even analysis, we demonstrated that the coordination factor may reverse expected gains from parallelization for some parameter combinations. In our experiments, the break-even point was between 110 and 160% of the original synchronization time and depended on the work load and the variability of the environment. Higher work load and higher variability lead to a lower break-even point which reflects a poorer performance of parallel designs. This effect can be explained with the excessive utilization of the synchronizing employee in the case of an additional coordination effort. This employee performs the coordination activities and becomes the bottleneck for the entire process.

Thus, the existence and strength of an additional coordination effort has to be estimated before (re-)engineering the process. One way to make parallel designs more favorable would be through the provision of additional resources for coordination activities. Another option is to automate coordination activities so that manual coordination efforts become obsolete.

7.3 Coordination activities are potential bottlenecks

The utilization of every involved employee is important for the overall process performance. If one employee is confronted with an overload situation, he becomes a bottleneck and the entire processing speed breaks down. As described above, this is especially true for the employee who performs the coordination activity. In our experiments with high work loads, a resource

utilization of more than 80% leads to extreme turnaround times and a noticeable slow-down of the overall process.

From the performance perspective it is important to avoid bottlenecks within the process and provide enough resources especially for performing the coordination task. On the other hand, additional coordination resources will increase the overall process costs. This trade-off is in line with the results of organizational literature (see paragraph 2, e.g. Malone and Smith 1988) and has to be clear to the process designer in order to make the right choice.

7.4 Parallel task-sequences are frequently more efficient than single parallel task

The scope of parallelization had an influence in our experiments on the overall processing speed. The performance gains were higher if sequences of tasks were parallelized in contrast to single parallel tasks. We scrutinized this effect regarding the parallelization of two-task-sequences compared to the parallelization of single tasks. The two-task-sequences caused a “shortening” of the process through an additional parallelization while maintaining one synchronization point only. Thus, additional parallel designs may be introduced while the coordination effort remains the same as in the case of parallelizing single tasks. In this context, we have to point out that this holds only true if the parallel task-sequences are balanced, which means that they have the same average processing time. Unbalanced task-sequences will reduce potential parallelization gains.

Therefore we conclude the following: if parallel job processing is considered as a viable design option for a business process, not necessarily single tasks should be subject of the parallel design but especially task-sequences are good candidates for process improvements.

8 Summary and future research

This paper discusses the favorability of parallel job processing compared with the sequential design of business processes. A synopsis of relevant application domains where parallel designs are used to speed-up business processes has demonstrated two important aspects: (a) the parallelization of tasks can speed-up business process, but (b) an additional coordination effort may reduce or even offset achievable performance gains.

The interdependency between parallel design gains and coordination effort has been evaluated in detail through an extensive simulation study. A real-world example of order processing in the construction supplies industry has been chosen as a reference scenario. Our experiments have shown some important effects with regard to parallel process designs: (a) parallelization gains are reduced with increasing process variability, (b) in some cases, an additional coordination effort of 10–30% is sufficient in order to neutralize gains from parallel designs, (c) the resource capacity for the coordination

activity is a bottleneck for the overall processing speed in high work load situations, and (d) the parallelization of multiple task leads to higher performance gains.

In order to confirm and enhance our results for other designs and environments, further research work is recommended. The analysis should be enhanced according to the following dimensions:

- Instead of using one employee per task, multiple employees could be assigned for one task.
- Besides exponential and triangular distributions, other theoretical and empirical stochastic distributions may be embedded in the analysis.
- In addition to the throughput time, other effects like the quality of the process outcome should be evaluated.
- Concerning the coordination effort we focused on the synchronization point. Further work should examine additional coordination activities during the execution of the parallel tasks.
- The coordination effort has to be categorized and quantified within empirical studies in different application domains and for different business processes in order to get a clearer understanding of the coordination volume in special environments.

The presented experimental analysis is not intended to be a comprehensive study of parallel designs within business processes but rather a starting point for further research in this area. Our experiments have demonstrated the importance and effects of coordination efforts within parallel designs. The presented insights should carefully be utilized when business processes are (re)designed within organizations.

References

- Achermann B (1998) Aufbau eines Outbound Telesales Centers in 12 Wochen. *HMD* 35(204):45–55
- Adler PS, Mandelbaum A, Nguyen V, Schwerer E (1995) From project to process management: an empirically-based framework for analyzing product development time. *Manage Sci* 41(3):458–484
- AitSahlia F, Johnson E, Will P (1995) Is concurrent engineering always a sensible proposition? *IEEE Trans Eng Manage* 42:166–170
- Amdahl G (1967) Validity of the single processor approach to achieving large scale computer capabilities. *Proc AFIPS Comput Conf* 30:483
- Barua A, Ravindran S (1996) Reengineering information sharing behaviour in organizations. *J Inf Technol* 11:261–272
- Blocher JD (1996) The customer order lead-time problem on parallel machines. *Nav Res Logist* 43:629–654
- Bukchin J, Rubinovitz J (2003) A weighted approach for assembly line design with station paralleling and equipment selection. *IIE Trans* 35:73–85
- Bulgren WG (1982) *Discrete system simulation*. Englewood Cliffs, New Jersey
- Buzacott JA (1990) Abandoning the moving assembly line: models of human operators and job sequencing. *Int J Prod Res* 28(5):821–839
- Buzacott JA (1996) Commonalities in reengineered business processes: models and issues. *Manage Sci* 42(5):768–782

- Choi SH, Kin JS (1998) A study on the measurement of comprehensive flexibility in manufacturing systems. *Comput Eng* 34(1):103–118
- Clark KB, Fujimoto T (1991) Product development performance. Harvard Business School, Boston
- Crowston KA (1997) Coordination theory approach to organizational process design. *Organ Sci* 8(2):157–175
- Cuiper R, Feldmann C, Rossgoderer U (1996) Rechnerunterstützte Parallelisierung von Konstruktion und Montageplanung. *Z wirtsch Fabr* 91(7):338–341
- Davenport TH (1993) Process innovation: reengineering work through information technology. Harvard Business School Press, Boston
- De Vreede GJ, Van Eijck DTT, Sol HG (1996) Dynamic modelling for re-engineering organizations. *INFOR* 34(1):28–42
- Edosomwan JA (1996) Organizational transformation and process reengineering. Delray Beach, St. Lucie Press, Florida
- Eversheim W (1995) Prozessorientierte Unternehmensorganisation. Springer, Berlin Heidelberg New York
- Farrington P, Nazemetz JW (1998) Evaluation of the performance domain of cellular and functional layouts. *Comput Eng* 34(1):91–101
- Gerpott TJ, Winzer P (1996) Simultaneous engineering: Kritische Analyse eines Planungs- und Organisationsansatzes zur Erfolgsverbesserung industrieller Produktinnovationen. *Zeitschrift Planung* (2):132–150
- Giannini PJ, Gruppe FH, Saholsky RM (1997) Reengineering through simulation modeling: optimizing a telephone ordering system at GPO. *Inf Syst Manage* 14(3):61–66
- Glücher R (1998) Durchsatzsteigerung in einem Image-Processing-System durch Parallelverarbeitung. *HMD* 35(203):58ff
- Goebel E (1996) Prozessorganisation—radikaler Neubeginn oder alte Wissensbestände im neuen Gewande. *Zeitschrift Planung* (4):309–318
- Griffin A (1993) Metrics for measuring product development cycle time. *J Prod Innov Manage* 10(1993):112–125
- Hächler G (1998) Die parallele Koordinationssprache ALWAN. *HMD* 35(203):38ff
- Hammer M (1990) Reengineering work: don't automate, obliterate. *Harvard Business Review* July–August:104–112
- Hammer M, Champy J (1993) Reengineering the Corporation: a Manifesto for Business Revolution. Harper Collins, New York
- Handfield RB (1994) Effects of concurrent engineering on make-to-order products. *IEEE Trans Eng Manage* 41:384–393
- Heiss HU (1994) Prozessorzuteilung in Parallelrechnern. BI-Wiss.-Verlag, Mannheim
- Housel T, Kanevsky VA (1995) Reengineering business processes: a complexity theory approach to value added. *INFOR* 33(4)
- IEEE (1995) Special issue on microprocessors. In: Proceedings of the IEEE 83(12)
- Jaenicke R (1998) Parallel processing: the future of embedded systems. *Can Electron* 13(3)
- Johnson RV (1983) A branch and bound algorithm for assembly line balancing problems with formulation irregularities. *Manage Sci* 29(11):1309–1324
- Kelton WD, Sadowski RP, Sadowski DA (1998) Simulation with ARENA. Mc Graw-Hill, Boston
- Kessler E, Chakrabarti AK (1999) Speeding up the pace of new product development. *J Prod Innov Manage* 16(3):231–247
- Kilman RH (1983) The costs of organization structure: dispelling the myths of independent divisions and organization-wide decision making. *Account Organ Soc* 8(4):341–357
- Kleinrock L (1975) Queuing systems. Wiley, New York
- Law AM et al (1991) Simulation modeling and analysis. McGraw-Hill, New York
- Liebl F (1995) Simulation. Oldenbourg, München
- Malone TW, Crowston K (1994) The interdisciplinary study of coordination. *ACM Comput Surv* 26(1):88–119
- Malone TW, Smith SA (1988) Modeling the performance of organizational structures. *Oper Res* 36(3):421–436
- Murmann PA (1994) Expected development time reductions in the German mechanical engineering industry. *J Prod Innov Manage* 11:236–252

- Nissen ME (1998) Redesigning reengineering through measurement-driven inference. *MIS Quarterly* December:509–534
- Ould M (1995) *Business processes: modelling and analysis for re-engineering and improvement*. Wiley, Chichester
- Piersma N, Romeijn HE (1996) Parallel machine scheduling: a probabilistic analysis. *Nav Res Logist* 43:897–916
- Pinto PA, Dannenbring DG, Khumawala BM (1975) A branch and bound algorithm for assembly line balancing with paralleling. *Int J Prod Res* 13(2):183–196
- Prietula MJ, Carley KM (1994) Computational organization theory: autonomous agents and emergent behavior. *J Organ Comput* 4(1):41–83
- Schmelzer HJ (1990) Steigerung der Effektivität und Effizienz durch Verkürzung von Entwicklungszeiten. In: Reichwald R, Schmelzer HJ (eds) *Durchlaufzeiten in der Entwicklung: Praxis des industriellen F&E Managements*, München, pp 27–64
- Schröder HH (1994) Die Parallelisierung von Forschungs- und Entwicklungs-(F&E)-Aktivitäten als Instrument zur Verkürzung der Projektdauer im Lichte des “Magischen Dreiecks” aus Projektdauer, Projektkosten und Projektergebnissen. In: Zahn E (ed) *Technologiemanagement und Technologien für das Management*, pp 289–323
- Seidmann A, Sundararajan A (1997) The Effects of Task and Information Asymmetry on Business Process Redesign. *Int J Prod Econ* 50(2–3):117–128
- Shanker K, Tzen YJJ (1985) A loading and dispatching problem in a random flexible manufacturing system. *Int J Prod Res* 23(3):579–595
- Sheu C, Babbar S (1996) A managerial assessment of the waiting-time performance for alternative service process designs. *Omega* 24(6):689–703
- Siemens C (1998) Paralele Programmierung—Nicht ohne Prozessor- und Rechner-technik. *HMD* 35(203):9–20
- Smith B (1999) Breaking through the I/O bottleneck. *Network World* 16(34)
- Stone HS (1987) *High performance computing architecture*. Addison-Wesley, Reading Mass
- Thompson JD (1967) *Organizations in action: social science bases of administrative theory*. McGraw-Hill, New York
- Trygg L (1993) Concurrent engineering practices in selected Swedish companies: a movement or an activity of the few? *J Prod Innov Manage* 10:403–415
- Walter ZD (1998) *Workflow management in business processes*. Dissertation University of Rochester, New York
- Wildemann H (1993) Just-In-Time in Forschung & Entwicklung und Konstruktion. *Zeitschrift Betriebswirtschaft* 63:1251–1270
- Zangl H (1985) *Durchlaufzeiten im Büro*. Schmidt, Berlin
- Zapf M (2001) *Gestaltung flexibler Kundeninteraktionsprozesse im Communication Center: Theoretische Grundlagen und experimentelle Analyse*, Dissertation, University of Bayreuth
- Zapf M, Heinzl A (2000) Evaluation of generic process design patterns: an experimental study. In: van der Aalst WMP, Desel J, Oberweis A (eds) *Business process management: models, techniques, and empirical studies*, LNCS 1806. Berlin
- Zapf M, Storch K (2001) Making simulation work for the organizational design of communication centers: challenges and practical experience. In: *Proceedings of the 2001 Summer Computer Simulation Conference*, Orlando